

UCD School of Physics



PHYC40600 - Advanced Laboratories
Acoustic Avoided Crossings

Dearbhail O'Mahony

October 1, 2026

Abstract

This report experimentally demonstrates the occurrence of avoided crossings in the resonance frequencies of an acoustic system, and compares this to the same effect as theoretically predicted in a simple quantum system of an infinite potential square well with a central delta potential. The resonance frequencies of two varyingly coupled tube sections, found from analysis of amplitude spectrums in the 1-2 kHz range, were plotted with changing length. These plots were found to clearly exhibit the effect of avoided crossings. The uncoupled, medium, and high coupling readings were found to closely match solutions to the quantum system.

Contents

1	Introduction	1
2	Methodology and Analysis	1
3	Results and Discussion	2
4	Conclusion	4
	References	5
	Appendices	6

1 Introduction

The phenomenon of avoided crossings occurs in the interaction of coupled systems, and is particularly well documented in quantum mechanics. It describes the change in the eigenvalues of a system, dependent on some coupling parameter, which will get closer together but never 'cross', or become equal. This can be seen in a wide range of quantum mechanical systems, and has many applications including in photonics and nanoscale devices, but can also be observed in the resonance frequencies of undamped mechanical systems.[1][2] Here, avoided crossings will be demonstrated with acoustic waves in two coupled cylindrical pipe sections.[3][4]

To compare this to an analogue quantum mechanical system, we can use the simple case of an infinite square potential well with a delta potential well in the centre. This gives two 'cavities', bounded by the square well wall and the delta well, similarly to the two pipe sections in the acoustic setup. Here we will have one section with an attached plunger that will be used to vary the length - therefore, we will take our quantum mechanical cavities to have one fixed length, L , and one variable length, $L + \Delta L$.

Taking our potential to then be $V(x) = -\alpha\delta(x - L)$ for $0 < x < 2L + \Delta L$, and infinite elsewhere, the coupling of this system is controlled by the parameter α , determining the strength of the delta potential well. As $\alpha \rightarrow \infty$ the system is uncoupled, and as α decreases, the probability of a particle traversing from one cavity to the other increases, therefore increasing the coupling of the system. Similarly, when the two sections of the acoustic system are separated by a solid diaphragm the system is uncoupled, and as the hole diameter in the diaphragm increases, more acoustic waves pass through, and the coupling is increased.

The quantum mechanical system is governed by the time-independent Schrödinger equation, given below in Equation (1), where $k = \sqrt{2mE}/\hbar$.

$$\frac{\delta^2 \psi}{\delta x^2} = k^2 \psi \quad (1)$$

This can be solved using the delta potential given above and appropriate boundary conditions to give Equation (2) below. Solving this equation for L and varying α will allow us to generate plots

comparable to those obtained experimentally.

$$\cot(kL) + \cot[k(L + \Delta L)] = \frac{2m\alpha}{\hbar^2 k} \quad (2)$$

The acoustic system is governed by the acoustic wave equation, which can be simplified to its time-independent form, with $k_s = 2\pi f/v$.

$$\frac{\delta^2 p}{\delta x^2} = k_s^2 p \quad (3)$$

Given the similar mathematical form of Equations (1) and (3), and the relationship given by k_s , we expect to obtain similar results in the resonance or eigen-frequencies in the acoustic system, and the eigen-energies in the quantum system, and to observe the effect of avoided crossings in both when they are plotted. [3]

2 Methodology and Analysis

Two PVC pipe sections were connected with a replaceable diaphragm in the centre. Each section had a microphone and a speaker inside, wired to a frequency generator and amplifier. The sections were separated by an aluminium disc with four hole diameter options.

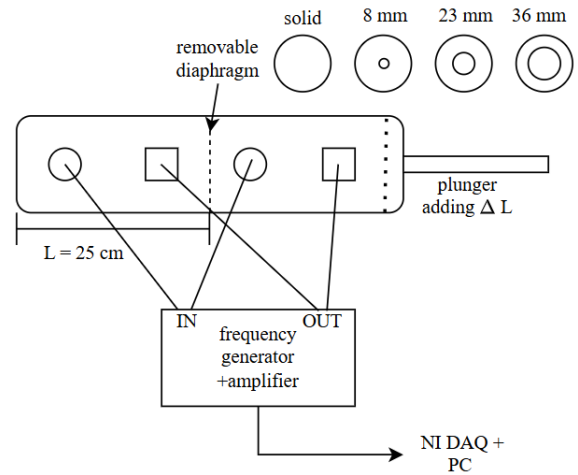


Figure 1: Experimental setup consisting of two tube sections, with $\circ$ = microphone, $\square$ = speaker, and a frequency generator/amplifier feeding to a NI DAQ and PC

With the equipment connected as described and in Figure (1), initial readings were read manually from the frequency generator for voltages between 1 and 5 V sent through the DAC at intervals of

0.5 V. These values were plotted and a linear fit was used as a calibration line to convert frequency to voltage for the proceeding analysis.

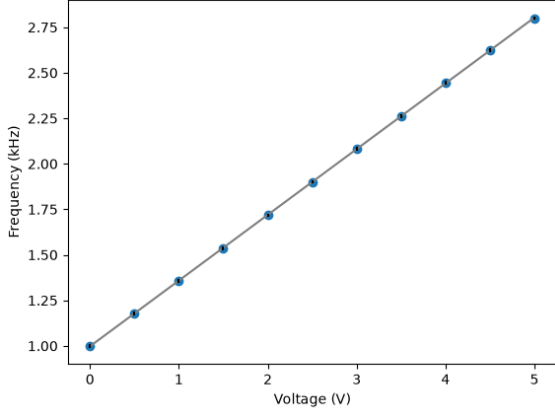


Figure 2: Calibration line to convert between voltage and frequency values. The linear fit shows very strong agreement and little error is introduced in this process.

The National Instruments DAQ DAC was then used to send voltages corresponding to a frequency range between 1 and 2 kHz to the frequency generator. Signals from the microphones were picked up, amplified, and sent back by the ADC. A signal was measured at 2 Hz intervals in this frequency range for ΔL between +6 cm and -4 cm.

The mean was subtracted to reduce the background in each signal, and then the RMS of each value was taken. The highest RMS amplitude was saved, so that an amplitude spectrum could be constructed over the frequency range. With these plotted, the frequency region in which the second harmonic fell was identified by eye in each channel for the solid disc (uncoupled system) and the resonance frequency was calculated. This was repeated for each length value, and resonance frequencies were plotted against length. By fitting a line to each channel, the point of intersection was identified and this was set as $\Delta L = 0$ going forward.

For each of the remaining diaphragms, two peaks were identified and tracked through the changing lengths, with the resonance frequency of each peak calculated at each value. As the two channels here provide information for a single coupled system, unlike in the solid diaphragm case, the channel from which data was taken was selected based on which exhibited the clearest peaks. Resonance frequencies of the two peaks were then plot-

ted against ΔL so that the effect of avoided crossings can be observed and for comparison with the theoretical prediction of the quantum mechanical system. For this purpose, equation (2) was solved using Scipy's *root_scalar* function, with the experimental results provided as guess solutions for k . Solutions were found at the same ΔL values, and with α coupling values chosen by eye to best match the experimental results.

3 Results and Discussion

Below is an example of the result of plotting an amplitude spectrum over a 1 to 2 kHz frequency range. These readings are taken using the smallest diaphragm hole diameter, and see pronounced resonance peaks can be observed at approximately 1150 and 1400 kHz. It was determined to be most accurate to track these peaks and select the 100-200 Hz range in which they fall by eye, which was then input into a function to compute the peak amplitude and corresponding resonance frequency.

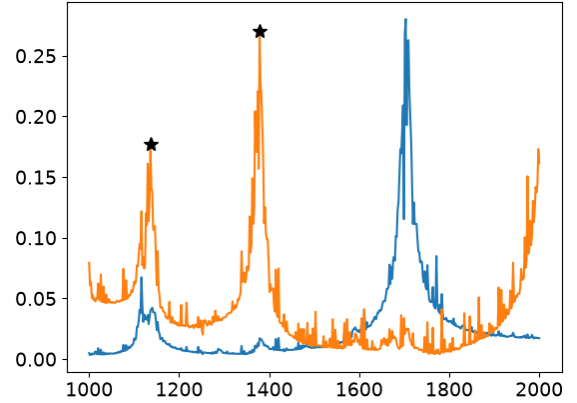


Figure 3: Amplitude spectrum generated from measurements with the smallest diaphragm size. Stars mark resonances that were tracked for plotting.

Some variation in peak size occurs at different resonances and across the two channels, which can likely be explained due to the relative position of the microphones in the tubes. In the larger diaphragm diameters, some of the higher frequency peaks were not well resolved above the noise level. This is a source of experimental error and may be due to non-optimal gain and volume settings on the frequency generator. For the purposes of obtaining complete resonance frequency plots from this data, a peak was assumed to be present for the calculations, and in most cases a small peak

could still be visually identified in the expected frequency range. However, this likely decreases the accuracy of these results, and points where this was an issue have been marked with a diamond in the resonance plots.

For the first set of results, taken with the solid diaphragm, a horizontal plot of constant resonance frequency at approximately 1.35 kHz is obtained for the fixed cavity (channel 1), and a straight line of increasing resonance frequency with plunger depth is found for the variable length cavity (channel 0). Linear fits of these plots are found to intercept at a plunger depth of 8.77 ± 0.02 cm, which will now be defined as $\Delta L = 0$.

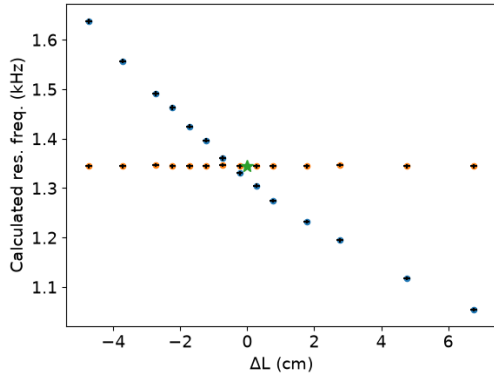


Figure 4: Results for uncoupled system (solid disc). Intercept ($\Delta L = 0$) point marked by green star as calculated using a linear fit.

Comparing this to the the results obtained from solving equation (2), using a very high α value to simulate an uncoupled system, we see a high level of agreement in the results.

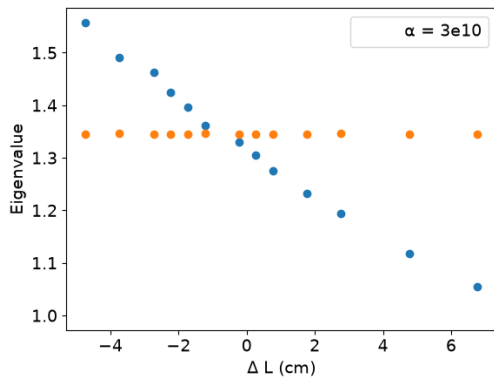


Figure 5: Eigenvalues of the uncoupled quantum system, corresponding to the solid disc results.

When the 8 mm diameter diaphragm is put in place, the system is now coupled and can no longer

be analysed in terms of the two distinct cavities. We now plot two successive resonances in the system, and track the location of these resonances over the ΔL range. When plotted, it is evident that these peaks exhibit the expected behaviour of avoided crossings. At this lowest coupling value the narrowest avoided crossing of the measured results can be observed, reaching a minimum crossing distance of 70 ± 5 Hz. Unfortunately theoretical values, in the form of solutions to equation (2), were not able to be obtained for this coupling amount such that they reliably resembled these results, so no comparison can be made.

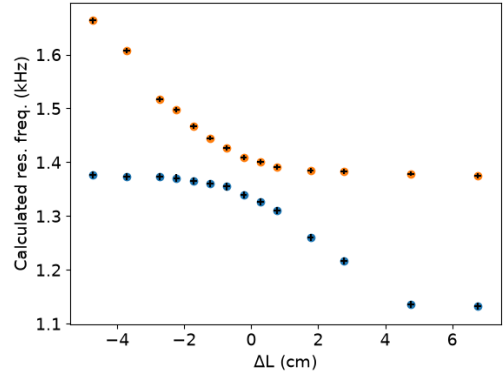


Figure 6: Results for lowest coupling (8 mm diaphragm)

Readings taken with a 23 mm diaphragm also show an avoided crossing between the two peaks, however the crossing is now wider and less pronounced due to the stronger coupling interaction. The minimum crossing distance is now 216 ± 5 Hz, significantly wider than in the previous case.

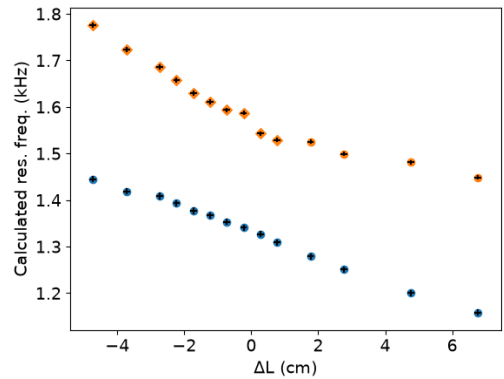


Figure 7: Results for medium coupling (23 mm diaphragm)

Despite small variations in the frequency range and shape of the plots, the theoretical results track well with the observed values for this coupling

level, and the minimum crossing of 212 Hz falls within the uncertainty range of the experimental value.

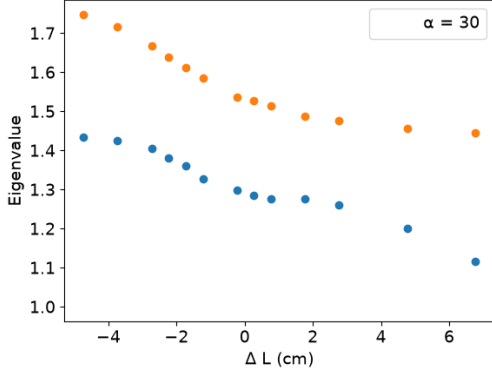


Figure 8: Results for medium coupling (23 mm diaphragm)

At a diaphragm diameter of 36 mm, the coupling of the system becomes so high that we no longer see an avoided crossing, and the consecutive resonance peaks remain at an approximately fixed distance from each other, with a mean separation of 309 ± 5 Hz. The plots now each resemble the variable section line in Figure (2).

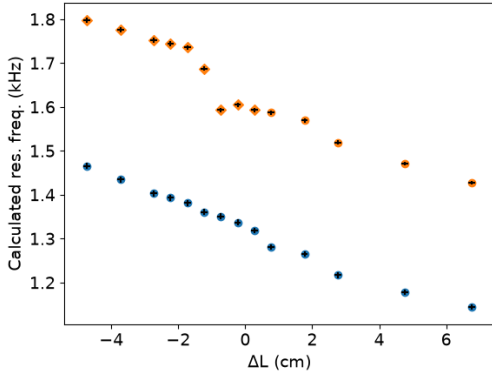


Figure 9: Results for highest coupling (36 mm diaphragm)

A similar behaviour is observed for a very low α value in the quantum mechanical system, where the eigenvalues are observed to remain separate over the range of ΔL values. The mean separation is the slightly lower value of 285 Hz.

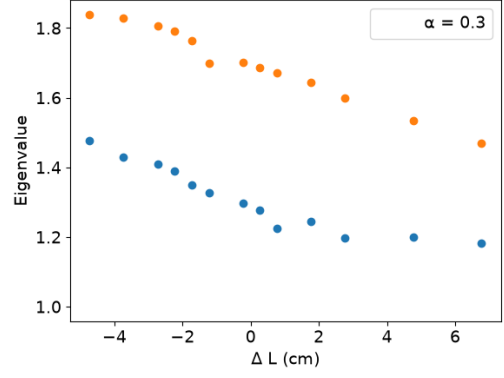


Figure 10: Theoretical results for the highest coupling

It is also evident that the two plots cover similar ranges for the eigen-/resonance frequencies, however the experimental plots have a slightly steeper slope. They therefore reach lower frequencies for each resonance at the high ΔL values, and there is some deviation here with the theoretical results which taper off at a marginally higher frequency.

4 Conclusion

The phenomenon of avoided crossings has been successfully demonstrated in an acoustic system consisting of two tube sections separated by a replaceable diaphragm, one of which had a variable length. By altering the hole size in the diaphragm, the coupling of the system was varied. Resonance frequencies were found for a range different length values at each coupling amount. When plotted, for the middle two coupling amounts an avoided crossing could be clearly observed in the resonance frequency values.

These plots were compared to theoretical values found by solving the time-independent Schrödinger for the relevant boundary conditions for a comparable quantum mechanical system of an infinite square potential well with a central delta potential. The resulting eigenfrequencies were found to correspond well with the experimental results and to show a similar avoided crossing effect. It can be concluded that the classical acoustic and quantum mechanical systems are a good analogue for each other in demonstrating the phenomenon of acoustic crossings.

References

- [1] Florian Jansen et al. “Avoided crossings in photonic crystal fibers”. In: *Optics Express* 19.14 (2011), p. 13578. DOI: 10.1364/oe.19.013578.
- [2] Jeeban K. Nayak et al. “Role of avoided crossing and weak value amplification on enhanced Faraday effect in magnetoplasmonic systems”. In: *Communications Physics* 4.1 (May 2021). DOI: 10.1038/s42005-021-00603-6. URL: <https://www.nature.com/articles/s42005-021-00603-6>.
- [3] William Newman, Alexandria Skinner, and Shawn A Hilbert. “An acoustic demonstration of an avoided crossing”. In: *American journal of physics* 85.11 (Nov. 2017), pp. 844–849. DOI: 10.1119/1.4996871.
- [4] Shawn A. Hilbert and Herman Batelaan. “Acoustic analog to quantum mechanical level splitting”. In: *American Journal of Physics* 75.11 (Nov. 2007), pp. 1003–1008. DOI: 10.1119/1.2772278.

Appendices

Appendix 1

Notebook used for analysis and data collection done within the lab.

```
In [3]: import matplotlib.pyplot as plt
import numpy as np
from time import sleep
import pandas as pd
```

<h4 id="Setting-up-DAQ-and-channels:">Setting up DAQ and channels:</h4>

```
In [2]: from pydaqmx_helper.adc import ADC
from pydaqmx_helper.dac import DAC
```

```
In [3]: myDAC = DAC(0)
```

```
In [4]: myADC = ADC()
myADC.addChannels([0, 1], minRange=0, maxRange=5)
```

Activated Channel 0
Activated Channel 1

<h4 id="Voltage/frequency-calibration:">Voltage/frequency calibration:</h4>

```
In [4]: cal_voltages = np.linspace(0, 5, 11)
cal_frequencies = np.array([1, 1.178, 1.357, 1.537, 1.717, 1.901, 2.082, 2.263, 2.444, 2.624, 2.8])#+- 0.0005
cal_voltages
```

```
Out[4]: array([0. , 0.5, 1. , 1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5, 5. ])
```

```
In [6]: from scipy.optimize import curve_fit

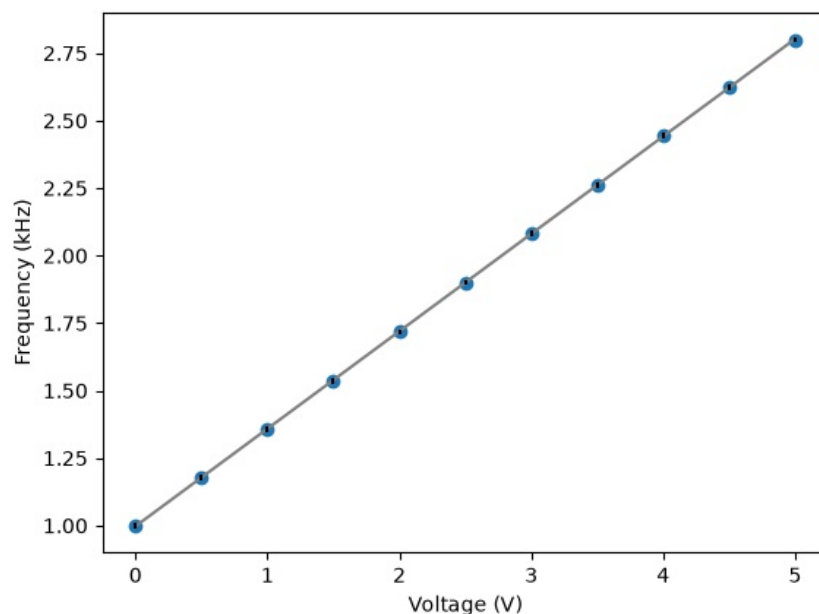
def lin(x, a, b):
    return a*x + b

popt, pcov = curve_fit(lin, cal_voltages, cal_frequencies)
a, b = popt
a_err, b_err = np.sqrt(np.diag(pcov))
print(a, b)
#covariance

a_upper = a+a_err
a_lower = a-a_err
b_upper = b+b_err
b_lower = b-b_err
```

0.36112727272727274 0.9974545454545448

```
In [12]: plt.scatter(cal_voltages, cal_frequencies)
plt.plot(cal_voltages, lin(cal_voltages, a, b), color='gray')
plt.xlabel('Voltage (V)')
plt.ylabel('Frequency (kHz)')
plt.errorbar(cal_voltages, cal_frequencies, xerr = 5/(2**12), yerr=0.01, fmt='none', color='black')
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/calibration.")
```



```
In [8]: def v2f(v):
    return lin(v, a, b)

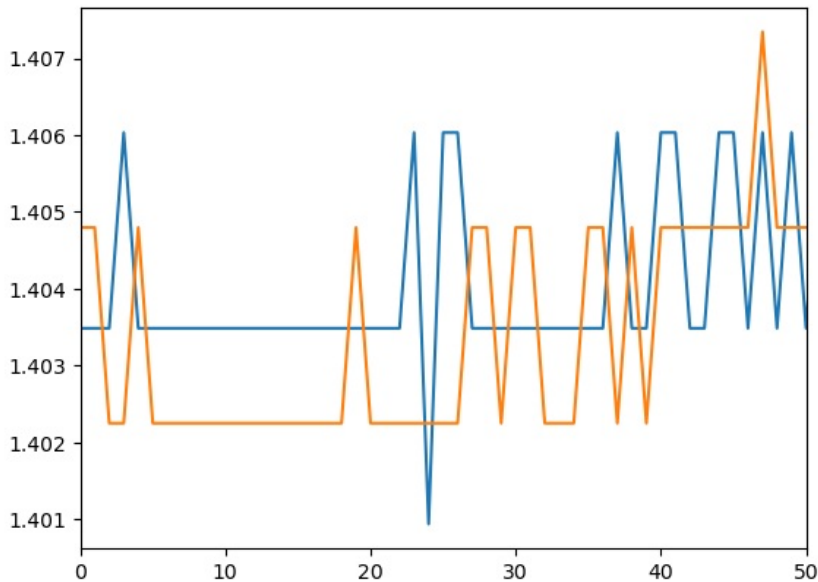
def f2v(f):
```

```
return (f - b)/a
```

```
In [9]: myDAC.writeVoltage(3)
sleep(0.1)
ADC_data = myADC.sampleVoltages(100, 5000)
ch0 = ADC_data[0]
ch1 = ADC_data[1]
```

```
In [10]: plt.plot(ch1)
plt.plot(ch0)
plt.xlim(0, 50)
```

```
Out[10]: (0.0, 50.0)
```



<h4 id="Measurements-&-initial-analysis:">Measurements & initial analysis:</h4>

```
In [11]: # defining a function that gets the RMS amplitude from measurements. this also finds resonances, however these
#recalculated in subsequent analysis outside of the lab
def get_amp_res(freq):
    amplitudes_ch0 = []
    amplitudes_ch1 = []

    for f in freq:
        myDAC.writeVoltage(f2v(f))
        sleep(0.1)
        ADC_data = myADC.sampleVoltages(100, 5000)
        ch0 = ADC_data[0]
        ch1 = ADC_data[1]

        ch0 = ch0 - np.mean(ch0)
        ch1 = ch1 - np.mean(ch1)

        squared_ch0 = []
        squared_ch1 = []
        for i in range(len(ch0)):
            squared_ch0.append(ch0[i]**2)
            squared_ch1.append(ch1[i]**2)

        amp_ch0 = np.sqrt(np.mean(squared_ch0))
        amp_ch1 = np.sqrt(np.mean(squared_ch1))

        amplitudes_ch0.append(amp_ch0)
        amplitudes_ch1.append(amp_ch1)

    res_ch0 = freq[np.where(amplitudes_ch0 == np.max(amplitudes_ch0))[0]]
    res_ch1 = freq[np.where(amplitudes_ch1 == np.max(amplitudes_ch1))[0]]

    return res_ch0, res_ch1, amplitudes_ch0, amplitudes_ch1
```

```
In [12]: frequency_range = np.linspace(1, 2, 500) #defining frequency range - 1 to 2 kHz
```

```
In [93]: #setting up dataframes to store values
columns_res = ('resonance_freq_ch0', 'resonance_freq_ch1')
med_resonance_df = pd.DataFrame(columns = columns_res, index=np.arange(0, 20, 1))
med_amplitude_df_ch0 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(0, 500, 1))
med_amplitude_df_ch1 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(0, 500, 1))
```

```

In [94]: i=0

In [112]: #this cell was run after each time the plunger was moved to add a value for the relevant delta L (intervals given)
res_ch0, res_ch1, amplitudes_ch0, amplitudes_ch1 = get_amp_res(frequency_range)

med_resonance_df.loc[i, 'resonance_freq_ch0'] = res_ch0
med_resonance_df.loc[i, 'resonance_freq_ch1'] = res_ch1

med_amplitude_df_ch0[i] = amplitudes_ch0
med_amplitude_df_ch1[i] = amplitudes_ch1
i+=1

In [118]: #saving values:
med_amplitude_df_ch0.to_csv('med_amp_ch0_3.csv', index=False)
med_amplitude_df_ch1.to_csv('med_amp_ch1_3.csv', index=False)
med_resonance_df.to_csv('med_res_3.csv', index=False)

In [74]: i=0

In [75]: #process repeated for each diaphragm size
big_resonance_df = pd.DataFrame(columns = columns_res, index=np.arange(0, 20, 1))
big_amplitude_df_ch0 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(0, 500, 1))
big_amplitude_df_ch1 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(0, 500, 1))

In [92]: res_ch0, res_ch1, amplitudes_ch0, amplitudes_ch1 = get_amp_res(frequency_range)

big_resonance_df.loc[i, 'resonance_freq_ch0'] = res_ch0
big_resonance_df.loc[i, 'resonance_freq_ch1'] = res_ch1

big_amplitude_df_ch0[i] = amplitudes_ch0
big_amplitude_df_ch1[i] = amplitudes_ch1
i+=1

In [96]: big_amplitude_df_ch0.to_csv('big_amp_ch0_3.csv', index=False)
big_amplitude_df_ch1.to_csv('big_amp_ch1_3.csv', index=False)
big_resonance_df.to_csv('big_res_3.csv', index=False)

In [24]: i=0

In [25]: small_resonance_df = pd.DataFrame(columns = columns_res, index=np.arange(0, 20, 1))
small_amplitude_df_ch0 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(1, 2, 0.002))
small_amplitude_df_ch1 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(1, 2, 0.002))

In [43]: res_ch0, res_ch1, amplitudes_ch0, amplitudes_ch1 = get_amp_res(frequency_range)

small_resonance_df.loc[i, 'resonance_freq_ch0'] = res_ch0
small_resonance_df.loc[i, 'resonance_freq_ch1'] = res_ch1

small_amplitude_df_ch0[i] = amplitudes_ch0
small_amplitude_df_ch1[i] = amplitudes_ch1
i+=1

In [41]: i

Out[41]: 13

In [45]: small_amplitude_df_ch0.to_csv('small_amp_ch0_3.csv', index=False)
small_amplitude_df_ch1.to_csv('small_amp_ch1_3.csv', index=False)
small_resonance_df.to_csv('small_res_3.csv', index=False)

In [114]: i=0

In [115]: solid_resonance_df = pd.DataFrame(columns = columns_res, index=np.arange(0, 20, 1))
solid_amplitude_df_ch0 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(0, 500, 1))
solid_amplitude_df_ch1 = pd.DataFrame(columns = np.arange(0, 20, 1), index=np.arange(0, 500, 1))

In [131]: res_ch0, res_ch1, amplitudes_ch0, amplitudes_ch1 = get_amp_res(frequency_range)

solid_resonance_df.loc[i, 'resonance_freq_ch0'] = res_ch0
solid_resonance_df.loc[i, 'resonance_freq_ch1'] = res_ch1

solid_amplitude_df_ch0[i] = amplitudes_ch0
solid_amplitude_df_ch1[i] = amplitudes_ch1
i+=1

In [132]: solid_amplitude_df_ch0.to_csv('solid_amp_ch0_3.csv', index=False)
solid_amplitude_df_ch1.to_csv('solid_amp_ch1_3.csv', index=False)
solid_resonance_df.to_csv('solid_res_3.csv', index=False)

```

In []:

Appendix 2

Notebook used for further data analysis.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import curve_fit, fsolve, brentq, root_scalar
from scipy.signal import find_peaks, savgol_filter, peak_widths
import scipy.constants as sc
```

<h4 id="Data-Preparation-&-Setup:">Data Preparation & Setup:</h4>

```
In [2]: #taking calibration voltages and frequencies from lab to use in frequency/voltage conversion:
cal_voltages = np.linspace(0, 5, 11)
cal_frequencies = np.array([1, 1.178, 1.357, 1.537, 1.717, 1.901, 2.082, 2.263, 2.444, 2.624, 2.8])
```

```
In [3]: def lin(x, a, b):
    return a*x + b

popt, pcov = curve_fit(lin, cal_voltages, cal_frequencies)
a, b = popt
a_err, b_err = np.sqrt(np.diag(pcov))
print(a, b)
```

0.36112727272727274 0.9974545454545448

```
In [4]: f2v_err = np.sqrt(a_err**2 + b_err**2)
```

```
In [5]: #calculating errors using calibration error and DAQ accuracy:
daq_err = 0.001
net_amp_err = np.sqrt(f2v_err**2 + daq_err**2)
net_amp_err
```

Out[5]: np.float64(0.0016200193823473926)

```
In [6]: #loading amplitude data taken in lab:
solid_amplitudes_ch0 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/solid_amplitudes_ch0.csv")
solid_amplitudes_ch1 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/solid_amplitudes_ch1.csv")

small_amplitudes_ch0 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/small_amplitudes_ch0.csv")
small_amplitudes_ch1 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/small_amplitudes_ch1.csv")

medium_amplitudes_ch0 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/medium_amplitudes_ch0.csv")
medium_amplitudes_ch1 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/medium_amplitudes_ch1.csv")

big_amplitudes_ch0 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/big_amplitudes_ch0.csv")
big_amplitudes_ch1 = pd.read_csv("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/big_amplitudes_ch1.csv")
```

```
In [7]: #tidying up (removing empty columns):
solid_amplitudes_ch0 = solid_amplitudes_ch0.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')
solid_amplitudes_ch1 = solid_amplitudes_ch1.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')

small_amplitudes_ch0 = small_amplitudes_ch0.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')
small_amplitudes_ch1 = small_amplitudes_ch1.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')

medium_amplitudes_ch0 = medium_amplitudes_ch0.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')
medium_amplitudes_ch1 = medium_amplitudes_ch1.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')

big_amplitudes_ch0 = big_amplitudes_ch0.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')
big_amplitudes_ch1 = big_amplitudes_ch1.drop(labels = ['15', '16', '17', '18', '19'], axis='columns')
```

```
In [8]: #defining frequency range (kHz) and setting index in amplitude datframes to frequency in Hz for ease of calculation:
ind = np.linspace(1000, 2000, 500)
frequency_range = np.linspace(1, 2, 500)
```

```
In [9]: solid_amplitudes_ch0_plot = solid_amplitudes_ch0.set_index(pd.Index(ind))
solid_amplitudes_ch1_plot = solid_amplitudes_ch1.set_index(pd.Index(ind))

small_amplitudes_ch0_plot = small_amplitudes_ch0.set_index(pd.Index(ind))
small_amplitudes_ch1_plot = small_amplitudes_ch1.set_index(pd.Index(ind))

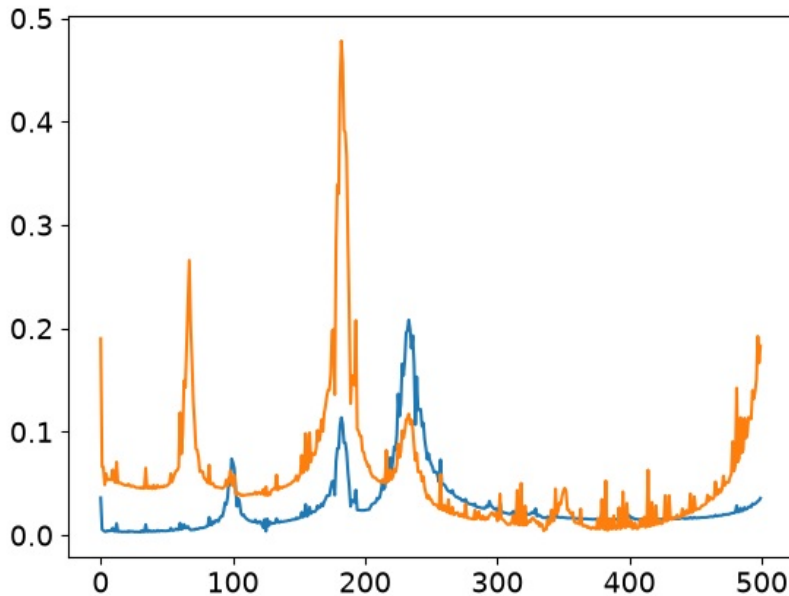
medium_amplitudes_ch0_plot = medium_amplitudes_ch0.set_index(pd.Index(ind))
medium_amplitudes_ch1_plot = medium_amplitudes_ch1.set_index(pd.Index(ind))

big_amplitudes_ch0_plot = big_amplitudes_ch0.set_index(pd.Index(ind))
big_amplitudes_ch1_plot = big_amplitudes_ch1.set_index(pd.Index(ind))
```

<h4 id="Finding-experimental-resonances:">Finding experimental resonances:</h4>

```
In [119]: #amplitude spectrums were used to visually identify the peak regions, so that upper and lower bounds could be placed on the data:
small_amplitudes_ch0['10'].plot()
small_amplitudes_ch1['10'].plot()
```

Out[119]: <Axes: >



```
In [11]: #defining functions that will get resonance values for the two channels in the uncoupled (solid diaphragm) setup
def get_res(amps, length, upper, lower):
    res_region = amps[length][(amps.index>lower)&(amps.index<upper)]
    res = frequency_range[np.where(amps==np.max(res_region))[0]]
    return res

In [12]: def get_res_ch1(amps, length): #ch1 has no delta L dependence as this is the fixed length section
    upper = 1400
    lower = 1200
    res_region = amps[length][(amps.index>lower)&(amps.index<upper)]
    res = frequency_range[np.where(amps==np.max(res_region))[0]][0]
    return res

In [13]: ch1_solid_res = []
for l in range(1, 15):
    res = get_res_ch1(solid_amplitudes_ch1_plot, str(l))
    ch1_solid_res.append(res)

In [14]: ch0_solid_res = []

In [15]: res_1 = get_res(solid_amplitudes_ch0_plot, '1', 1200, 1000)[0]
ch0_solid_res.append(res_1)

In [16]: res_2 = get_res(solid_amplitudes_ch0_plot, '2', 1200, 1000)[0]
ch0_solid_res.append(res_2)

In [17]: res_3 = get_res(solid_amplitudes_ch0_plot, '3', 1300, 1100)[0]
ch0_solid_res.append(res_3)

In [18]: res_4 = get_res(solid_amplitudes_ch0_plot, '4', 1300, 1100)[0]
ch0_solid_res.append(res_4)

In [19]: res_5 = get_res(solid_amplitudes_ch0_plot, '5', 1400, 1200)[0]
ch0_solid_res.append(res_5)

In [20]: res_6 = get_res(solid_amplitudes_ch0_plot, '6', 1400, 1200)[0]
ch0_solid_res.append(res_6)

In [21]: res_7 = get_res(solid_amplitudes_ch0_plot, '7', 1400, 1200)[0]
ch0_solid_res.append(res_7)

In [22]: res_8 = get_res(solid_amplitudes_ch0_plot, '8', 1400, 1200)[0]
ch0_solid_res.append(res_8)

In [23]: res_9 = get_res(solid_amplitudes_ch0_plot, '9', 1500, 1300)[0]
ch0_solid_res.append(res_9)

In [24]: res_10 = get_res(solid_amplitudes_ch0_plot, '10', 1500, 1300)[0]
ch0_solid_res.append(res_10)

In [25]: res_11 = get_res(solid_amplitudes_ch0_plot, '11', 1600, 1400)[0]
```

```

ch0_solid_res.append(res_11)

In [26]: res_12 = get_res(solid_amplitudes_ch0_plot, '12', 1600, 1400)[0]
ch0_solid_res.append(res_12)

In [27]: res_13 = get_res(solid_amplitudes_ch0_plot, '13', 1700, 1500)[0]
ch0_solid_res.append(res_13)

In [28]: res_14 = get_res(solid_amplitudes_ch0_plot, '14', 1700, 1500)[0]
ch0_solid_res.append(res_14)

In [29]: #delta L values as taken in lab
lengths = np.array([2, 4, 6, 7, 8, 8.5, 9, 9.5, 10, 10.5, 11, 11.5, 12.5, 13.5])

In [31]: #linear fits to solid diaphragm values so that intercept can be found
def lin(x, a, b):
    return a*x + b

popt, pcov = curve_fit(lin, lengths, ch0_solid_res)
a_solid_res_ch0, b_solid_res_ch0 = popt
a_err_solid_res_ch0, b_err_solid_res_ch0 = np.sqrt(np.diag(pcov))

popt, pcov = curve_fit(lin, lengths, ch1_solid_res)
a_solid_res_ch1, b_solid_res_ch1 = popt
a_err_solid_res_ch1, b_err_solid_res_ch1 = np.sqrt(np.diag(pcov))

x_int = (b_solid_res_ch0 - b_solid_res_ch1)/(a_solid_res_ch1 - a_solid_res_ch0)
y_int = lin(x_int, a_solid_res_ch0, b_solid_res_ch0)

In [32]: #error on intercept value:
np.sqrt(a_err_solid_res_ch0**2 + b_err_solid_res_ch0**2)

Out[32]: np.float64(0.023320023818359528)

In [33]: x_int

Out[33]: np.float64(8.774403768562905)

In [35]: plt.rcParams.update({'font.size': 14})

In [36]: plt.scatter(-lengths+8.77, ch0_solid_res, s=20)
plt.scatter(-lengths+8.77, ch1_solid_res, s=20)

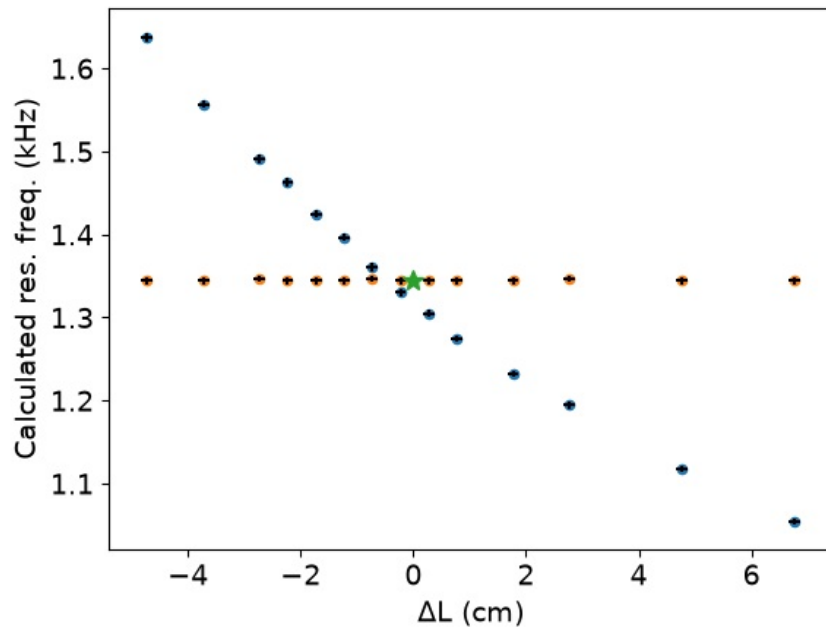
plt.errorbar(-lengths+8.77, ch0_solid_res, xerr = 0.1, yerr = np.sqrt((np.array(ch0_solid_res)*(0.0016/0.5))**2)
plt.errorbar(-lengths+8.77, ch1_solid_res, xerr = 0.1, yerr = np.array(ch1_solid_res)*(0.0016/0.5), fmt='none',

# plt.plot(-lengths+8.77, lin(-lengths+8.77, -a_solid_res_ch0, b_solid_res_ch0))
# plt.plot(-lengths+8.77, lin(-lengths+8.77, a_solid_res_ch1, b_solid_res_ch1))

plt.plot(0, y_int, color='tab:green', marker='*', ms = 10, label = 'Intercept: ΔL=0 @ '+str(np.round(x_int, 2))

plt.xlabel('ΔL (cm)')
plt.ylabel('Calculated res. freq. (kHz)')
#plt.legend()
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/solid_disc.p

```



```
In [37]: #finding peaks for example plot included in report
amps = small_amplitudes_ch1_plot
length = '2'

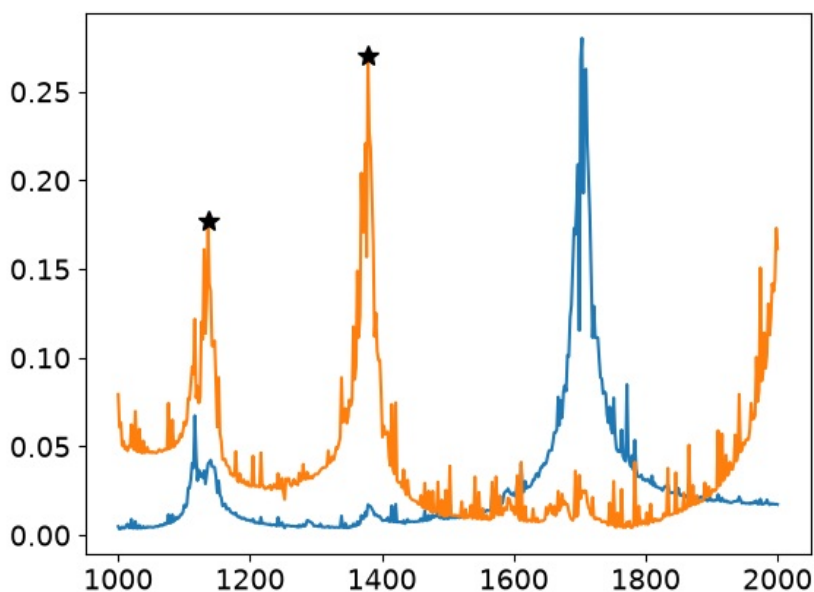
lower1 = 1100
upper1 = 1200
lower2 = 1300
upper2 = 1450

res1_region = amps[length][(amps.index>lower1)&(amps.index<upper1)]
a1 = np.max(res1_region)
res1 = frequency_range[np.where(amps==a1)[0]]

res2_region = amps[length][(amps.index>lower2)&(amps.index<upper2)]
a2 = np.max(res2_region)
res2 = frequency_range[np.where(amps==a2)[0]]
```

```
In [38]: small_amplitudes_ch0_plot['2'].plot()
small_amplitudes_ch1_plot['2'].plot()
plt.plot(res1*1000, a1, marker = '*', color='black', ms=10)
plt.plot(res2*1000, a2, marker = '*', color='black', ms=10)

plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/amp_spectrum
```



```
In [39]: #defining a function to get resonances for the coupled system. using this function,
#the same process was repeated as in the solid diaphragm case for the three
#diaphragm hole sizes
def get_res_coupled(amps, length, lower1, upper1, lower2, upper2):
    res1_region = amps[length][(amps.index>lower1)&(amps.index<upper1)]
    res1 = frequency_range[np.where(amps==np.max(res1_region))[0]]
```

```

res2_region = amps[length][ (amps.index>lower2)&(amps.index<upper2)]
res2 = frequency_range[np.where(amps==np.max(res2_region))[0]]
return res1, res2

```

```

In [40]: small_r1 = []
        small_r2 = []

```

```

In [41]: res1_1, res2_1 = get_res_coupled(small_amplitudes_ch1_plot, '1', 1100, 1200, 1300, 1450)
        small_r1.append(res1_1[0])
        small_r2.append(res2_1[0])

```

```

In [42]: res1_2, res2_2 = get_res_coupled(small_amplitudes_ch1_plot, '2', 1100, 1200, 1300, 1450)
        small_r1.append(res1_2[0])
        small_r2.append(res2_2[0])

```

```

In [43]: res1_3, res2_3 = get_res_coupled(small_amplitudes_ch1_plot, '3', 1150, 1250, 1300, 1450)
        small_r1.append(res1_3[0])
        small_r2.append(res2_3[0])

```

```

In [44]: res1_4, res2_4 = get_res_coupled(small_amplitudes_ch1_plot, '4', 1200, 1300, 1300, 1450)
        small_r1.append(res1_4[0])
        small_r2.append(res2_4[0])

```

```

In [45]: res1_5, res2_5 = get_res_coupled(small_amplitudes_ch1_plot, '5', 1250, 1350, 1350, 1450)
        small_r1.append(res1_5[0])
        small_r2.append(res2_5[0])

```

```

In [46]: res1_6, res2_6 = get_res_coupled(small_amplitudes_ch1_plot, '6', 1250, 1350, 1350, 1450)
        small_r1.append(res1_6[0])
        small_r2.append(res2_6[0])

```

```

In [47]: res1_7, res2_7 = get_res_coupled(small_amplitudes_ch1_plot, '7', 1300, 1400, 1400, 1500)
        small_r1.append(res1_7[0])
        small_r2.append(res2_7[0])

```

```

In [48]: res1_8, res2_8 = get_res_coupled(small_amplitudes_ch1_plot, '8', 1300, 1400, 1400, 1500)
        small_r1.append(res1_8[0])
        small_r2.append(res2_8[0])

```

```

In [49]: res1_9, res2_9 = get_res_coupled(small_amplitudes_ch1_plot, '9', 1300, 1400, 1400, 1500)
        small_r1.append(res1_9[0])
        small_r2.append(res2_9[0])

```

```

In [50]: res1_10, res2_10 = get_res_coupled(small_amplitudes_ch1_plot, '10', 1300, 1400, 1400, 1500)
        small_r1.append(res1_10[0])
        small_r2.append(res2_10[0])

```

```

In [51]: res1_11, res2_11 = get_res_coupled(small_amplitudes_ch1_plot, '11', 1350, 1450, 1450, 1550)
        small_r1.append(res1_11[0])
        small_r2.append(res2_11[0])

```

```

In [52]: res1_12, res2_12 = get_res_coupled(small_amplitudes_ch1_plot, '12', 1350, 1450, 1450, 1550)
        small_r1.append(res1_12[0])
        small_r2.append(res2_12[0])

```

```

In [53]: res1_13, res2_13 = get_res_coupled(small_amplitudes_ch1_plot, '13', 1350, 1450, 1550, 1650)
        small_r1.append(res1_13[0])
        small_r2.append(res2_13[0])

```

```

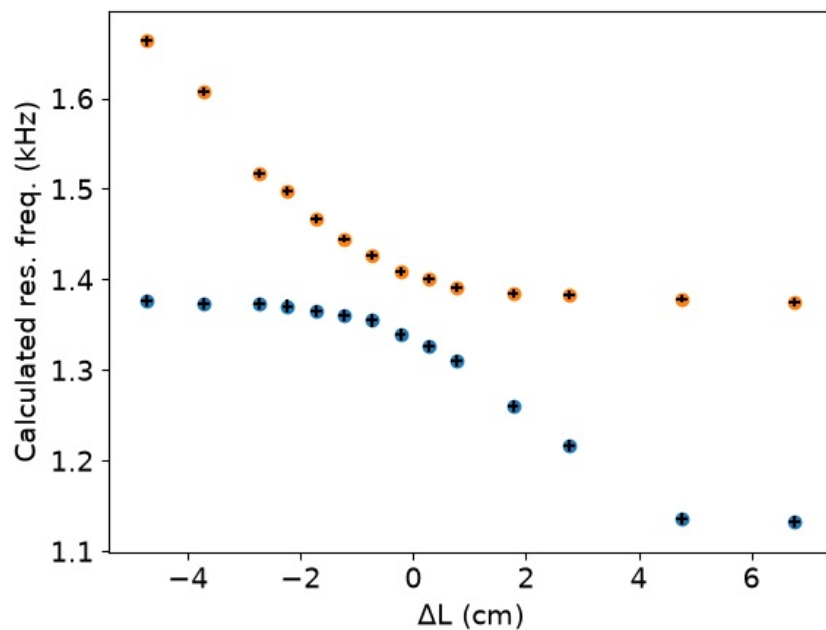
In [54]: res1_14, res2_14 = get_res_coupled(small_amplitudes_ch0_plot, '14', 1300, 1400, 1600, 1800)
        small_r1.append(res1_14[0])
        small_r2.append(res2_14[0])

```

```

In [55]: plt.scatter(-lengths+8.77, small_r1)
        plt.scatter(-lengths+8.77, small_r2)
        plt.errorbar(-lengths+8.77, small_r1, xerr = 0.1, yerr = np.sqrt((np.array(small_r1)*(0.0016/0.5))**2 + (0.005)**2))
        plt.errorbar(-lengths+8.77, small_r2, xerr = 0.1, yerr = np.array(small_r2)*(0.0016/0.5), fmt='none', color='blue')
        plt.xlabel('ΔL (cm)')
        plt.ylabel('Calculated res. freq. (kHz)')
        plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/small_disc.png")

```



```
In [127]: #finding avoided crossing width:
small_widths = []
small_width_errs = []
for i, val in enumerate(small_r1):
    w = small_r2[i] - val
    small_widths.append(w)
    small_width_errs.append(np.sqrt((np.array(val)*(0.0016/0.5))**2 + (0.005)**2 + np.array(small_r2[i])*(0.0016/0.5))**2))
small_min_crossing = np.min(small_widths)
small_min_crossing
```

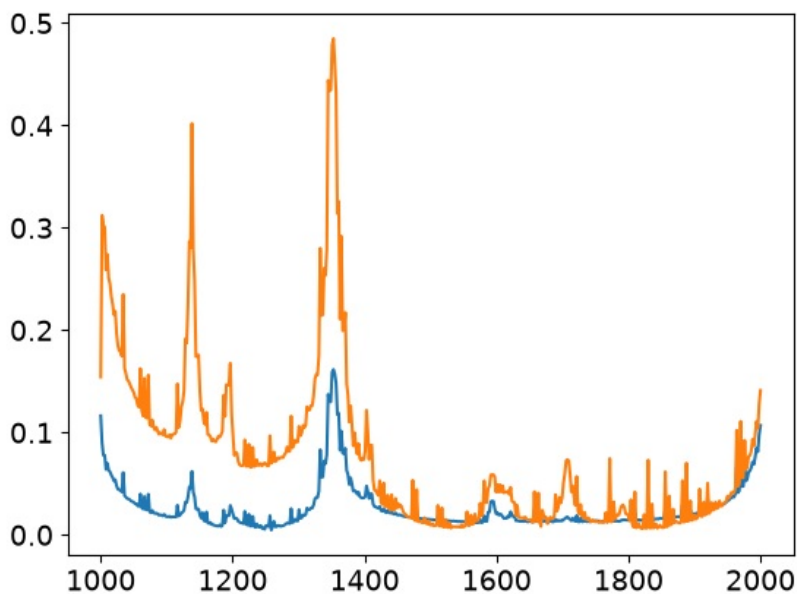
```
Out[127]: np.float64(0.0701402805611222)
```

```
In [128]: crossing_err = small_width_errs[np.where(small_widths == small_min_crossing)[0][0]]
crossing_err
```

```
Out[128]: np.float64(0.004576567097160251)
```

```
In [56]: medium_amplitudes_ch0_plot['8'].plot()
medium_amplitudes_ch1_plot['8'].plot()
```

```
Out[56]: <Axes: >
```



```
In [57]: med_r1 = []
med_r2 = []
```

```
In [58]: res1_1, res2_1 = get_res_coupled(medium_amplitudes_ch1_plot, '1', 1100, 1200, 1400, 1500)
med_r1.append(res1_1[0])
med_r2.append(res2_1[0])
```

```
In [59]: res1_2, res2_2 = get_res_coupled(medium_amplitudes_ch1_plot, '2', 1150, 1250, 1450, 1550)
```

```

med_r1.append(res1_2[0])
med_r2.append(res2_2[0])

In [60]: res1_3, res2_3 = get_res_coupled(medium_amplitudes_ch1_plot, '3', 1200, 1300, 1450, 1500)
med_r1.append(res1_3[0])
med_r2.append(res2_3[0])

In [61]: res1_4, res2_4 = get_res_coupled(medium_amplitudes_ch1_plot, '4', 1200, 1300, 1500, 1600)
med_r1.append(res1_4[0])
med_r2.append(res2_4[0])

In [62]: res1_5, res2_5 = get_res_coupled(medium_amplitudes_ch1_plot, '5', 1200, 1400, 1500, 1550)
med_r1.append(res1_5[0])
med_r2.append(res2_5[0])

In [63]: res1_6, res2_6 = get_res_coupled(medium_amplitudes_ch1_plot, '6', 1200, 1400, 1500, 1550)
med_r1.append(res1_6[0])
med_r2.append(res2_6[0])

In [64]: res1_7, res2_7 = get_res_coupled(medium_amplitudes_ch1_plot, '7', 1200, 1400, 1550, 1650)
med_r1.append(res1_7[0])
med_r2.append(res2_7[0])

In [65]: res1_8, res2_8 = get_res_coupled(medium_amplitudes_ch1_plot, '8', 1200, 1400, 1550, 1650)
med_r1.append(res1_8[0])
med_r2.append(res2_8[0])

In [66]: res1_9, res2_9 = get_res_coupled(medium_amplitudes_ch1_plot, '9', 1200, 1400, 1600, 1700)
med_r1.append(res1_9[0])
med_r2.append(res2_9[0])

In [67]: res1_10, res2_10 = get_res_coupled(medium_amplitudes_ch1_plot, '10', 1200, 1400, 1600, 1700)
med_r1.append(res1_10[0])
med_r2.append(res2_10[0])

In [68]: res1_11, res2_11 = get_res_coupled(medium_amplitudes_ch0_plot, '11', 1350, 1450, 1600, 1700)
med_r1.append(res1_11[0])
med_r2.append(res2_11[0])

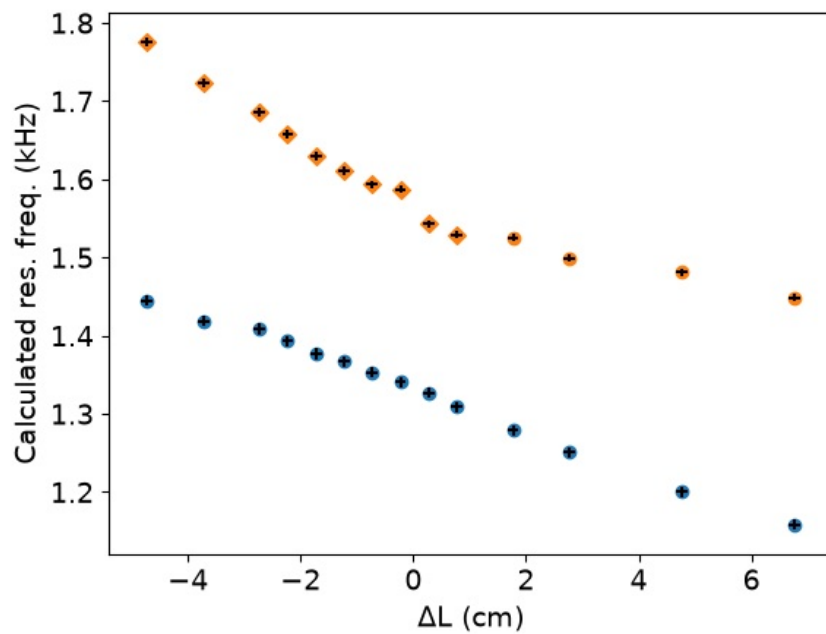
In [69]: res1_12, res2_12 = get_res_coupled(medium_amplitudes_ch0_plot, '12', 1350, 1450, 1650, 1750)
med_r1.append(res1_12[0])
med_r2.append(res2_12[0])

In [70]: res1_13, res2_13 = get_res_coupled(medium_amplitudes_ch0_plot, '13', 1350, 1450, 1700, 1800)
med_r1.append(res1_13[0])
med_r2.append(res2_13[0])

In [71]: res1_14, res2_14 = get_res_coupled(medium_amplitudes_ch0_plot, '14', 1350, 1450, 1700, 1800)
med_r1.append(res1_14[0])
med_r2.append(res2_14[0])

In [72]: plt.scatter(-lengths + 8.77, med_r1)
plt.scatter(-lengths[0:4] + 8.77, med_r2[0:4])
plt.scatter(-lengths[4:] + 8.77, med_r2[4:], color='tab:orange', marker='D')
plt.errorbar(-lengths+8.77, med_r1, xerr = 0.1, yerr = np.sqrt((np.array(med_r1)*(0.0016/0.5))**2 + (0.005)**2))
plt.errorbar(-lengths+8.77, med_r2, xerr = 0.1, yerr = np.array(med_r2)*(0.0016/0.5), fmt='none', color='black')
plt.xlabel('ΔL (cm)')
plt.ylabel('Calculated res. freq. (kHz)')
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/medium_disc.

```



```
In [123]: med_widths = []
med_width_errs = []
for i, val in enumerate(med_r1):
    w = med_r2[i] - val
    med_widths.append(w)
    med_width_errs.append(np.sqrt((np.array(val)*(0.0016/0.5))**2 + (0.005)**2 + np.array(med_r2[i])*(0.0016/0.5)))
med_min_crossing = np.min(med_widths)
med_min_crossing
```

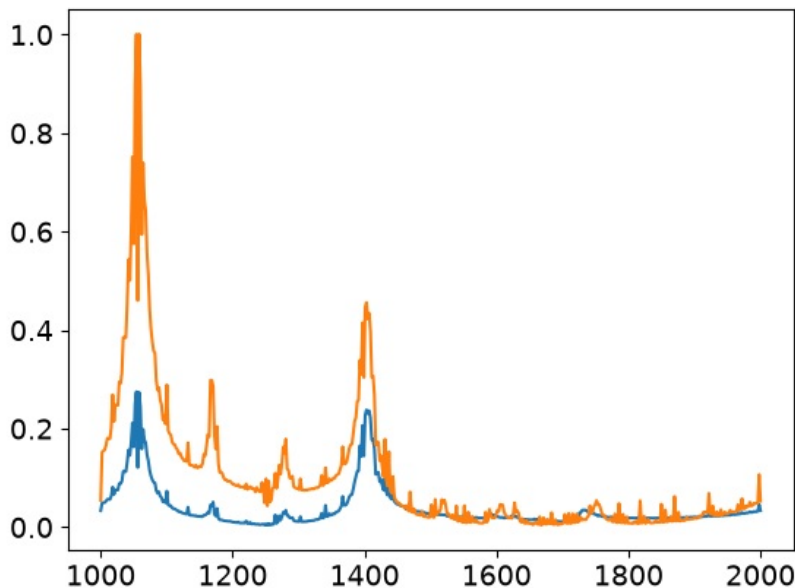
```
Out[123]: np.float64(0.21643286573146292)
```

```
In [126]: crossing_err = med_width_errs[np.where(med_widths == med_min_crossing)[0][0]]
crossing_err
```

```
Out[126]: np.float64(0.005005898243621512)
```

```
In [73]: big_amplitudes_ch0_plot['12'].plot()
big_amplitudes_ch1_plot['12'].plot()
```

```
Out[73]: <Axes: >
```



```
In [74]: big_r1 = []
big_r2 = []
```

```
In [75]: res1_1, res2_1 = get_res_coupled(big_amplitudes_ch1_plot, '1', 1100, 1200, 1400, 1500)
big_r1.append(res1_1[0])
big_r2.append(res2_1[0])
```

```
In [76]: res1_2, res2_2 = get_res_coupled(big_amplitudes_ch1_plot, '2', 1150, 1250, 1450, 1550)
big_r1.append(res1_2[0])
```

```

big_r2.append(res2_2[0])

In [77]: res1_3, res2_3 = get_res_coupled(big_amplitudes_ch1_plot, '3', 1200, 1300, 1500, 1600)
big_r1.append(res1_3[0])
big_r2.append(res2_3[0])

In [78]: res1_4, res2_4 = get_res_coupled(big_amplitudes_ch1_plot, '4', 1200, 1300, 1500, 1600)
big_r1.append(res1_4[0])
big_r2.append(res2_4[0])

In [79]: res1_5, res2_5 = get_res_coupled(big_amplitudes_ch1_plot, '5', 1250, 1350, 1550, 1650)
big_r1.append(res1_5[0])
big_r2.append(res2_5[0])

In [80]: res1_6, res2_6 = get_res_coupled(big_amplitudes_ch1_plot, '6', 1250, 1350, 1550, 1650)
big_r1.append(res1_6[0])
big_r2.append(res2_6[0])

In [81]: res1_7, res2_7 = get_res_coupled(big_amplitudes_ch1_plot, '7', 1300, 1400, 1550, 1650)
big_r1.append(res1_7[0])
big_r2.append(res2_7[0])

In [82]: res1_8, res2_8 = get_res_coupled(big_amplitudes_ch1_plot, '8', 1300, 1400, 1550, 1600)
big_r1.append(res1_8[0])
big_r2.append(res2_8[0])

In [83]: res1_9, res2_9 = get_res_coupled(big_amplitudes_ch1_plot, '9', 1300, 1400, 1650, 1700)
big_r1.append(res1_9[0])
big_r2.append(res2_9[0])

In [84]: res1_10, res2_10 = get_res_coupled(big_amplitudes_ch1_plot, '10', 1350, 1450, 1700, 1800)
big_r1.append(res1_10[0])
big_r2.append(res2_10[0])

In [85]: res1_11, res2_11 = get_res_coupled(big_amplitudes_ch1_plot, '11', 1350, 1450, 1700, 1800)
big_r1.append(res1_11[0])
big_r2.append(res2_11[0])

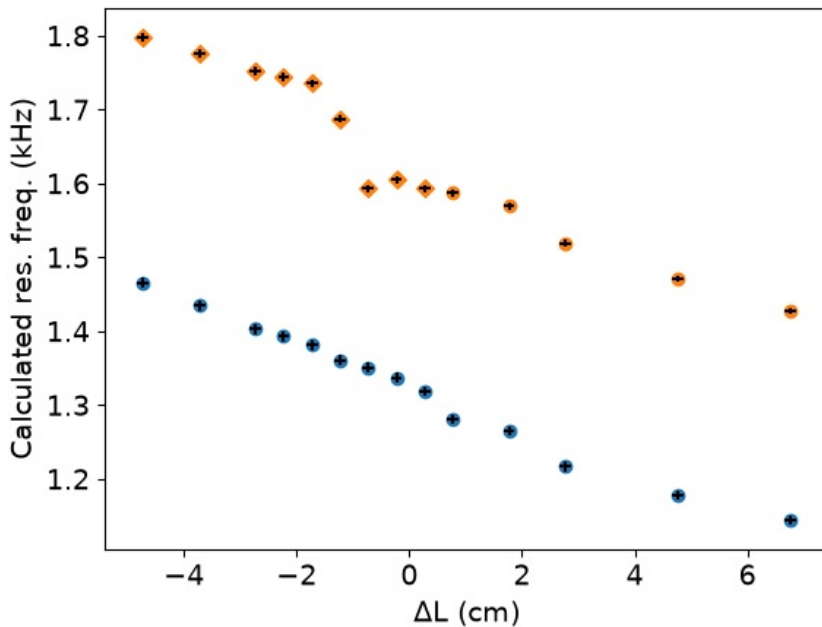
In [86]: res1_12, res2_12 = get_res_coupled(big_amplitudes_ch1_plot, '12', 1350, 1450, 1700, 1800)
big_r1.append(res1_12[0])
big_r2.append(res2_12[0])

In [87]: res1_13, res2_13 = get_res_coupled(big_amplitudes_ch0_plot, '13', 1400, 1500, 1700, 1800)
big_r1.append(res1_13[0])
big_r2.append(res2_13[0])

In [88]: res1_14, res2_14 = get_res_coupled(big_amplitudes_ch0_plot, '14', 1400, 1500, 1700, 1800)
big_r1.append(res1_14[0])
big_r2.append(res2_14[0])

In [89]: plt.scatter(-lengths+8.77, big_r1)
plt.scatter(-lengths[0:5]+8.77, big_r2[0:5], color='tab:orange')
plt.scatter(-lengths[5:15]+8.77, big_r2[5:15], color='tab:orange', marker='D')
plt.errorbar(-lengths+8.77, big_r1, xerr = 0.1, yerr = np.sqrt((np.array(big_r1)*(0.0016/0.5))**2 + (0.005)**2))
plt.errorbar(-lengths+8.77, big_r2, xerr = 0.1, yerr = np.array(big_r2)*(0.0016/0.5), fmt='none', color='black')
plt.xlabel('ΔL (cm)')
plt.ylabel('Calculated res. freq. (kHz)')
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/big_disc.png")

```



```
In [129]: big_widths = []
big_width_errs = []
for i, val in enumerate(big_r1):
    w = big_r2[i] - val
    big_widths.append(w)#
    big_width_errs.append(np.sqrt((np.array(val)*(0.0016/0.5))**2 + (0.005)**2 + np.array(big_r2[i])*(0.0016/0.5)))
big_min_crossing = np.min(big_widths)
big_min_crossing
```

```
Out[129]: np.float64(0.24248496993987967)
```

```
In [130]: big_max_crossing = np.max(big_widths)
big_max_crossing
```

```
Out[130]: np.float64(0.35470941883767537)
```

```
In [132]: big_mean_w = np.mean(big_widths)
big_mean_w
```

```
Out[132]: np.float64(0.3091898081878041)
```

```
In [133]: mean_w_err = np.mean(big_width_errs)
mean_w_err
```

```
Out[133]: np.float64(0.005292182850028714)
```

<h4 id="Solving-the-quantum-mechanical-system:">Solving the quantum mechanical system:</h4>

```
In [90]: #defining functions to convert between k (eigenvalue) and f (frequency)
```

```
def freq(k):
    return (k*343)/(2*np.pi)

def kval(f):
    return (2*np.pi*f)/343
```

```
In [91]: #delta L in metres to use for equation
```

```
dl_m = [0.02, 0.04, 0.06, 0.07, 0.08, 0.085, 0.09, 0.1, 0.105, 0.11, 0.115, 0.125, 0.135]

dl_eq = []
for dl in dl_m:
    delta = -dl + 0.0877
    dl_eq.append(delta)
```

```
In [92]: #defining function and parameters
```

```
L = 0.25 #fixed tube length measured to be 25 cm
a_val = [3*10**10, 100, 30, 0.3]

def qm_func(k, dL, a):
    return 1/np.tan(k*L) + 1/np.tan(k*(L + dL)) - 2*a/k
```

```
In [139]: #solving for uncoupled system
```

```
f1_sols = []
```

```

f2_sols = []

k1_vals = []
k2_vals = []

i = 0

alpha = 3*10**10
delta_fixed = 0

for delta in dl_eq:
    k1 = root_scalar(qm_func, args=(delta, alpha), x0 = kval(ch0_solid_res[i]*1000), xtol=30)
    k2 = root_scalar(qm_func, args=(delta_fixed, alpha), x0 = kval(ch1_solid_res[i]*1000), xtol=30)

    k1 = k1['root']
    k2 = k2['root']

    k1_vals.append(k1)
    k2_vals.append(k2)

    f1 = freq(k1)
    f2 = freq(k2)

    f1_sols.append((f1/2)/1000)
    f2_sols.append((f2/2)/1000)

    i+=1

```

```

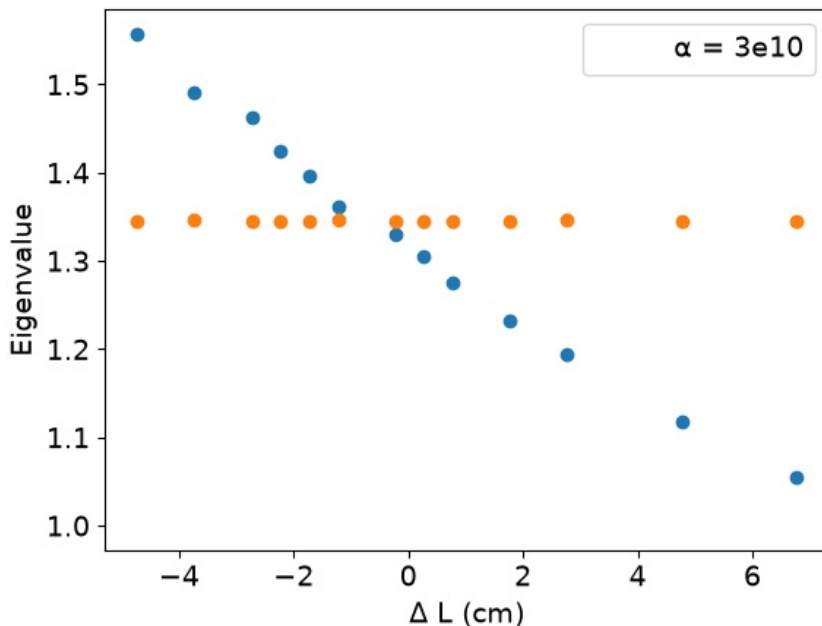
In [140...] dl_plot = []
for dl in dl_eq:
    dl = dl*100
    dl_plot.append(dl)

```

```

In [142...] plt.scatter(dl_plot, f1_sols)
plt.scatter(dl_plot, f2_sols)
plt.plot(0, 1, marker='None', linestyle = 'None', label = ' $\alpha = 3e10$ ')
plt.legend()
plt.xlabel('Δ L (cm)')
plt.ylabel('Eigenvalue')
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/theoretical_

```



```

In [109...] #solving for low coupling - this does not produce a plot that resembles the expected results and was not included

f1_sols = []
f2_sols = []

k1_vals = []
k2_vals = []

i = 0

alpha = 100

for delta in dl_eq:
    k1 = root_scalar(qm_func, args=(delta, alpha), x0 = kval(small_r1[i]*1000), xtol=5)
    k2 = root_scalar(qm_func, args=(delta, alpha), x0 = kval(small_r2[i]*1000), xtol=5)

```

```

k1 = k1['root']
k2 = k2['root']

k1_vals.append(k1)
k2_vals.append(k2)

f1 = freq(k1)
f2 = freq(k2)

f1_sols.append(f1/1000)
f2_sols.append(f2/1000)

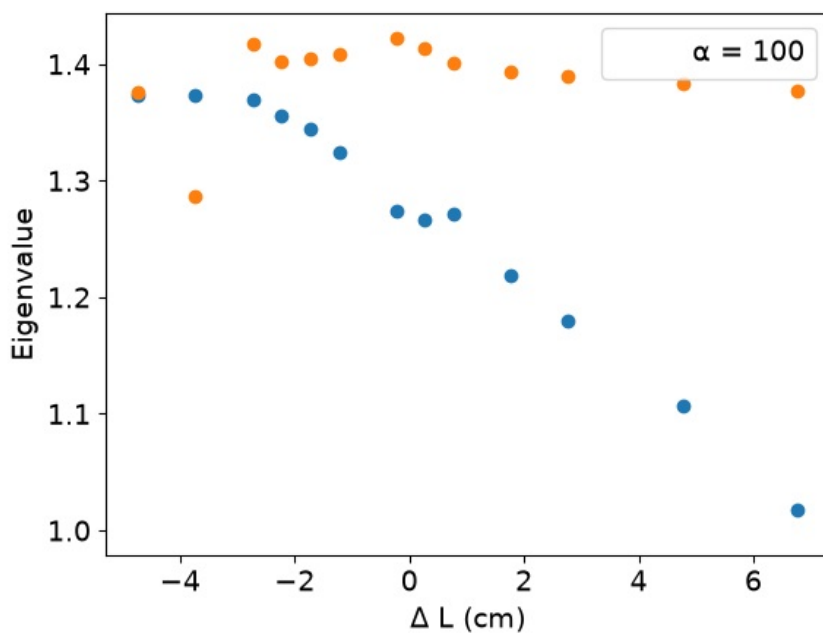
i+=1

```

```

In [110]: plt.scatter(dl_plot, f1_sols)
plt.scatter(dl_plot, f2_sols)
plt.plot(0, 1, marker='None', linestyle = 'None', label = ' $\alpha = 100$ ')
plt.legend()
# plt.ylim(1000, 2000)
plt.xlabel('Δ L (cm)')
plt.ylabel('Eigenvalue')
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/theoretical_

```



```

In [134]: #solving for medium coupling

f1_sols = []
f2_sols = []

k1_vals = []
k2_vals = []

i = 0
alpha = 30

for delta in dl_eq:
    k1 = root_scalar(qm_func, args=(delta, alpha), x0 = kval(med_r1[i]*1000), xtol=10)
    k2 = root_scalar(qm_func, args=(delta, alpha), x0 = kval(med_r2[i]*1000), xtol=10)

    k1 = k1['root']
    k2 = k2['root']

    k1_vals.append(k1)
    k2_vals.append(k2)

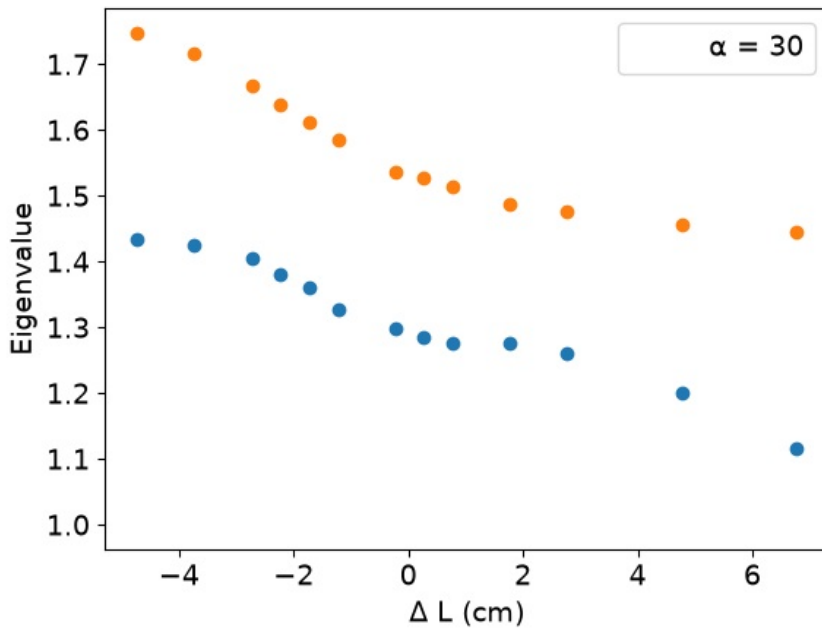
    f1 = freq(k1)
    f2 = freq(k2)

    f1_sols.append(f1/1000)
    f2_sols.append(f2/1000)

    i+=1

```

```
In [117... plt.scatter(dl_plot, f1_sols)
plt.scatter(dl_plot, f2_sols)
plt.xlabel('Δ L (cm)')
plt.ylabel('Eigenvalue')
plt.plot(0, 1, marker='None', linestyle = 'None', label = 'α = 30')
plt.legend()
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/theoretical_
```



```
In [136... med_th_widths = []
for i, val in enumerate(f1_sols):
    w = f2_sols[i] - val
    med_th_widths.append(w)#
med_th_min_crossing = np.min(med_th_widths)
med_th_min_crossing
```

```
Out[136... np.float64(0.21215277964546253)
```

```
In [137... #solving for high coupling

f1_sols = []
f2_sols = []

k1_vals = []
k2_vals = []

i = 0

alpha = 0.3

for delta in dl_eq:
    k1 = root_scalar(qm_func, args=(delta, alpha), x0 = kval(big_r1[i]*1000), xtol=10)
    k2 = root_scalar(qm_func, args=(delta, alpha), x0 = kval(big_r2[i]*1000), xtol=10)

    k1 = k1['root']
    k2 = k2['root']

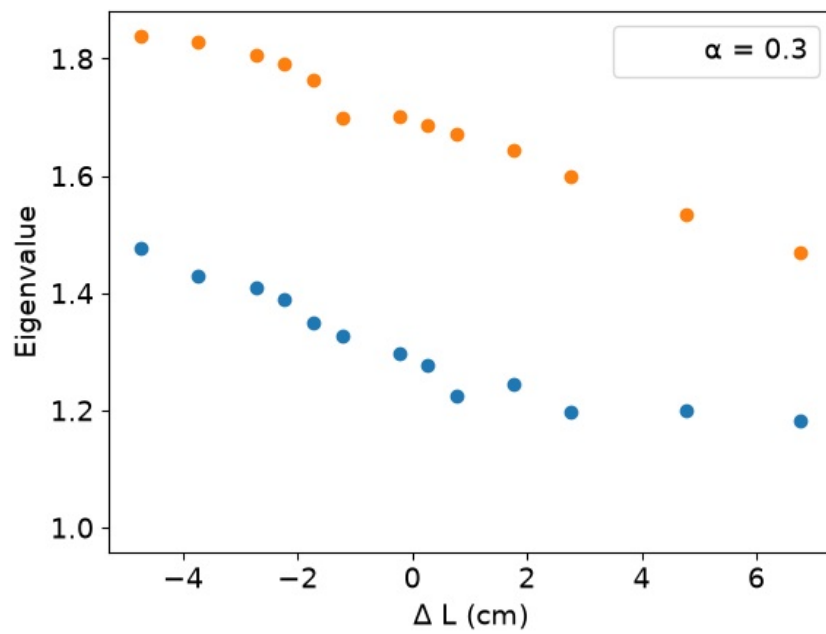
    k1_vals.append(k1)
    k2_vals.append(k2)

    f1 = freq(k1)
    f2 = freq(k2)

    f1_sols.append(f1/1000)
    f2_sols.append(f2/1000)

    i+=1
```

```
In [112... plt.scatter(dl_plot, f1_sols)
plt.scatter(dl_plot, f2_sols)
plt.xlabel('Δ L (cm)')
plt.ylabel('Eigenvalue')
plt.plot(0, 1, marker='None', linestyle = 'None', label = 'α = 0.3')
plt.legend()
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/acoustic_avoided_crossings/figures/theoretical_
```



```
In [138.. big_th_widths = []
for i, val in enumerate(f1_sols):
    w = f2_sols[i] - val
    big_th_widths.append(w)#
big_th_mean_sep = np.min(big_th_widths)
big_th_mean_sep
```

```
Out[138.. np.float64(0.2852860966157722)
```

```
In [ ]:
```